

1. Simple HTML Helpers

Simple HTML Helpers are methods that generate HTML elements like text boxes, checkboxes, etc. **They do not have direct links to a model (not strongly typed).**

Example:

```
@Html.TextBox("Name", "John Doe")    <!-- Simple HTML Helper
-->
@Html.Password("Password")           <!-- Simple HTML Helper
-->
@Html.CheckBox("RememberMe", true)   <!-- Simple HTML Helper
-->
```

Explanation:

- `@Html.TextBox("Name", "John Doe")`: Creates a text box with the name `Name` and default value `John Doe`.
- `@Html.Password("Password")`: Creates a password input field with the name `Password`.
- `@Html.CheckBox("RememberMe", true)`: Creates a checkbox with the name `RememberMe` and is checked by default.

Rendered HTML Output:

```
<input type="text" name="Name" value="John Doe" />
<input type="password" name="Password" />
<input type="checkbox" name="RememberMe" checked="checked"
/>
```

2. Strongly Typed HTML Helpers

Strongly Typed Helpers are used when you're **working with a model and want to generate HTML elements** that are **bound to properties of a model**. These helpers provide **better IntelliSense and compile-time checking**.

Example:

Assume you have a **model** like this:

```
public class LoginModel
{
    public string Username { get; set; }
    public string Password { get; set; }
    public bool RememberMe { get; set; }
}
```

In your **Razor view**:

```
@model LoginModel

@Html.LabelFor(m => m.Username)           <!-- Strongly Typed
Helper -->
@Html.EditorFor(m => m.Username)           <!-- Strongly Typed
Helper -->
@Html.LabelFor(m => m.Password)           <!-- Strongly Typed
Helper -->
@Html.PasswordFor(m => m.Password)         <!-- Strongly Typed
Helper -->
@Html.CheckBoxFor(m => m.RememberMe)       <!-- Strongly Typed
Helper -->
```

Explanation:

- `@Html.LabelFor(m => m.Username)`: Generates a `<label>` for the `Username` property.
- `@Html.EditorFor(m => m.Username)`: Generates an appropriate HTML element (e.g., text box) for the `Username` property.
- `@Html.PasswordFor(m => m.Password)`: Generates a password input field bound to the `Password` property.
- `@Html.CheckBoxFor(m => m.RememberMe)`: Generates a checkbox input bound to the `RememberMe` property.

Rendered HTML Output:

```
<label for="Username">Username</label>
<input type="text" name="Username" value="" />
<label for="Password">Password</label>
<input type="password" name="Password" />
<input type="checkbox" name="RememberMe" />
```

3. Tag Helpers

Tag Helpers in ASP.NET Core MVC are a more modern way of working with HTML elements. They are processed on the server side and enhance the HTML tags with additional attributes like `asp-for`, `asp-action`, etc., which simplify dynamic content generation and provide cleaner syntax.

Example:

```
<form asp-action="Login" method="post">
    <input asp-for="Username" class="form-control" /> <!--
Tag Helper -->
    <input asp-for="Password" class="form-control" /> <!--
Tag Helper -->
    <input asp-for="RememberMe" type="checkbox" /> <!--
Tag Helper -->
    <button type="submit">Login</button>
</form>
```

Explanation:

- `asp-for="Username"`: Binds the `input` element to the `Username` property of the model. It's equivalent to `@Html.EditorFor(m => m.Username)` but using the tag helper syntax.
- `asp-for="Password"`: Binds the `input` element to the `Password` property of the model.
- `asp-for="RememberMe"`: Binds the `input` element to the `RememberMe` property and automatically renders it as a checkbox.

Rendered HTML Output:

```
<form action="/Home/Login" method="post">
  <input type="text" name="Username" class="form-control"
/>
  <input type="password" name="Password"
class="form-control" />
  <input type="checkbox" name="RememberMe" />
  <button type="submit">Login</button>
</form>
```

Mix Example Model

```
public class LoginModel
{
    public string Name { get; set; }
    public string Password { get; set; }
    public bool RememberMe { get; set; }
}
```

Key Differences:

1. Simple HTML Helper

Simple HTML Helpers use **string names** to identify model properties.

```
@model LoginModel
```

```
<form asp-action="Login" method="post">
```

```
    @Html.TextBox("Name", Model.Name)
```

```
    @Html.Password("Password", Model.Password)
```

```
    @Html.CheckBox("RememberMe", Model.RememberMe)
```

```
    <button type="submit">Login</button>
```

```
</form>
```

How it works

```
@Html.TextBox("Name", Model.Name)
```

Here:

"Name" → field name

Model.Name → field value

It generates approximately:

```
<input type="text" name="Name" value="Kailas" />
```

2. Strongly Typed HTML Helper

Strongly Typed Helpers use a **lambda expression** to select the model property.

```
@model LoginModel
```

```
<form asp-action="Login" method="post">
```

```
    @Html.TextBoxFor(x => x.Name)
```

```
    @Html.PasswordFor(x => x.Password)
```

```
    @Html.CheckBoxFor(x => x.RememberMe)
```

```
    <button type="submit">Login</button>
```

```
</form>
```

For example:

```
@Html.TextBoxFor(x => x.Name)
```

means:

Create a textbox for the **Name** property of my **LoginModel**.

You don't write "**Name**" as a string.

Visual Studio knows that **Name** is a property of **LoginModel**.

It generates approximately:

```
<input type="text" name="Name" value="Kailas" />
```

3. Tag Helper

Tag Helpers use HTML elements with **asp-for**.

```
@model LoginModel
```

```
<form asp-action="Login" method="post">
```

```
    <input asp-for="Name" class="form-control" />
```

```
    <input asp-for="Password" class="form-control" />
```

```
    <input asp-for="RememberMe" type="checkbox" />
```

```
    <button type="submit">Login</button>
```

```
</form>
```

For example:

```
<input asp-for="Name" />
```

means:

Create an HTML input that is connected to the **Name** property of the model.

It generates approximately:

```
<input type="text"
      id="Name"
      name="Name"
      value="Kailas" />
```

Feature	Simple HTML Helpers	Strongly Typed HTML Helpers	Tag Helpers
Binding to Model	Does not bind to a model	Binds to model properties using lambda expressions	Binds to model properties using <code>asp-for</code>
Server-Side Processing	No server-side processing, purely client-side	Server-side processing for model binding	Server-side processing with dynamic URL creation
Code Completion (IntelliSense)	Limited IntelliSense support	Full IntelliSense support for model properties	Full IntelliSense support for Razor attributes
Readability	Less readable, often using strings	More readable and maintainable with model bindings	Clean, concise, and easy-to-read syntax
Use Case	Simple scenarios where you don't need model binding	Scenarios where you need to bind input fields to model data	Modern web applications with dynamic content generation
Example Usage	<code>@Html.TextBox("Username", "user123")</code>	<code>@Html.TextBoxFor(m => m.Username)</code>	<code><input asp-for="Username" /></code>

Conclusion:

- **Simple HTML Helpers** are quick to use but offer minimal functionality and no model binding. They are suitable for simple applications where model binding isn't necessary.
- **Strongly Typed Helpers** are ideal for applications that require binding form elements to model properties, providing compile-time validation and IntelliSense support.
- **Tag Helpers** are the modern and recommended approach in ASP.NET Core MVC. They offer clean syntax, dynamic content generation, and enhanced features, making them the preferred option in newer projects.

1. HTML Helpers

HTML Helpers are methods that return strings of HTML content. These methods are usually invoked in Razor views to create form elements, links, and other HTML controls dynamically. There are two types of HTML helpers:

- **Standard HTML Helpers:** These are basic helpers that help generate standard HTML elements. For example:
 - `@Html.TextBox("name")`: Generates an HTML `<input>` text box element.
 - `@Html.DropDownList("name", selectList)`: Creates a `<select>` element (dropdown).
 - `@Html.ActionLink("linkText", "action", "controller")`: Generates an anchor (`<a>`) tag linking to a specific action and controller.
- **Strongly Typed HTML Helpers:** These helpers are bound to model properties, making them type-safe, and reducing errors. Examples:
 - `@Html.TextBoxFor(m => m.PropertyName)`: Creates a text input field for a model property.
 - `@Html.EditorFor(m => m.PropertyName)`: Renders an appropriate input field based on the property type.

- **@Html.ValidationMessageFor(m => m.PropertyName):** Displays validation messages for a model property.

Example:

```
@model MyApp.Models.User

<form>
    @Html.LabelFor(m => m.Username)
    @Html.TextBoxFor(m => m.Username)
    @Html.ValidationMessageFor(m => m.Username)

    <input type="submit" value="Submit" />
</form>
```

2. Tag Helpers

Tag Helpers are a more modern and declarative way to generate HTML in Razor views. They use HTML-like syntax and allow you to apply logic directly to HTML elements using attributes. Tag Helpers have a more natural and intuitive feel compared to HTML Helpers, and they are especially useful when generating form elements and interacting with ASP.NET Core features. Tag Helpers are applied to HTML elements in Razor views by using attributes. For example, instead of calling a method like `@Html.TextBoxFor()`, you can apply a Tag Helper to a normal HTML `<input>` tag.

Commonly Used Tag Helpers:

- **Form Tag Helpers:** Bind the form to a specific controller and action.
 - Example: `<form asp-action="SubmitForm" asp-controller="Home">`
- **Input Tag Helpers:** Automatically bind the `<input>` element to a model property.
 - Example: `<input asp-for="Username" />` generates an `<input>` element for the `Username` property.
- **Anchor Tag Helpers:** Create anchor tags (`<a>`) that generate links to specific actions and controllers.

- Example: `<a asp-controller="Home" asp-action="Index">Home</a>` generates a link to the "Index" action of the "Home" controller.
- **Validation Tag Helpers:** Display validation messages based on model validation.
 - Example: `<span asp-validation-for="Username"></span>`

Example:

```
@model MyApp.Models.User
```

```
<form asp-action="SubmitForm" asp-controller="Home">
  <label asp-for="Username"></label>
  <input asp-for="Username" />
  <span asp-validation-for="Username"></span>

  <input type="submit" value="Submit" />
</form>
```

Key Differences:

- **HTML Helpers:** Method-based, less declarative, more flexible when it comes to dynamic logic in Razor views.
- **Tag Helpers:** Attribute-based, more declarative, and provide a more HTML-like syntax with enhanced readability and maintainability.

Which to Choose?

- **HTML Helpers** are still supported but are considered more old-fashioned compared to Tag Helpers.
- **Tag Helpers** are generally preferred in modern ASP.NET Core applications due to their more readable syntax and better integration with HTML and Razor.

//Insert

```
@* <div class="form-group">
  <label asp-for="Gender"></label>
  <select asp-for="Gender" class="form-control">
    <option value="">Select Gender</option>
    <option value="M">Male</option>
    <option value="F">Female</option>
  </select>
  <span asp-validation-for="Gender" class="text-danger"></span>
</div> *@

<div class="form-group">
  <label asp-for="Gender" class="control-label col-md-2"></label>
  <div class="col-md-10">
```

```

        <div class="form-check form-check-inline">
            <input class="form-check-input" type="radio" asp-for="Gender"
value="M" />
            <label class="form-check-label">Male</label>
        </div>
        <div class="form-check form-check-inline">
            <input class="form-check-input" type="radio" asp-for="Gender"
value="F" />
            <label class="form-check-label">Female</label>
        </div>

        <span asp-validation-for="Gender" class="text-danger"></span>
    </div>
</div>

```

//update

```

<div class="form-group">
    <label asp-for="Gender" class="control-label col-md-2"></label>
    <div class="col-md-10">
        <div class="form-check form-check-inline">
            @{
                if (Model.Gender == "M")
                {
                    <input class="form-check-input" type="radio"
asp-for="Gender" value="M" checked />
                }
                else
                {
                    <input class="form-check-input" type="radio"
asp-for="Gender" value="M" />
                }
            }
            <label class="form-check-label">Male</label>
        </div>
        <div class="form-check form-check-inline">
            @{
                if (Model.Gender == "F")
                {
                    <input class="form-check-input" type="radio"
asp-for="Gender" value="F" checked />
                }
                else
                {
                    <input class="form-check-input" type="radio"
asp-for="Gender" value="F" />
                }
            }
            <label class="form-check-label">Female</label>
        </div>

        <span asp-validation-for="Gender" class="text-danger"></span>
    </div>

```

```
</div>  
</div>
```

```
@* <div class="form-group">  
  <label asp-for="Gender"></label>  
  <select asp-for="Gender" class="form-control">  
    <option value="">Select Gender</option>  
  
    @if (Model.Gender == "M")  
    {  
  
      <option value="M" selected>Male</option>  
      <option value="F">Female</option>  
    }  
    else if (Model.Gender == "F")  
    {  
      <option value="M">Male</option>  
      <option value="F" selected>Female</option>  
    }  
  </select>  
  <span asp-validation-for="Gender" class="text-danger"></span>  
</div> *@
```